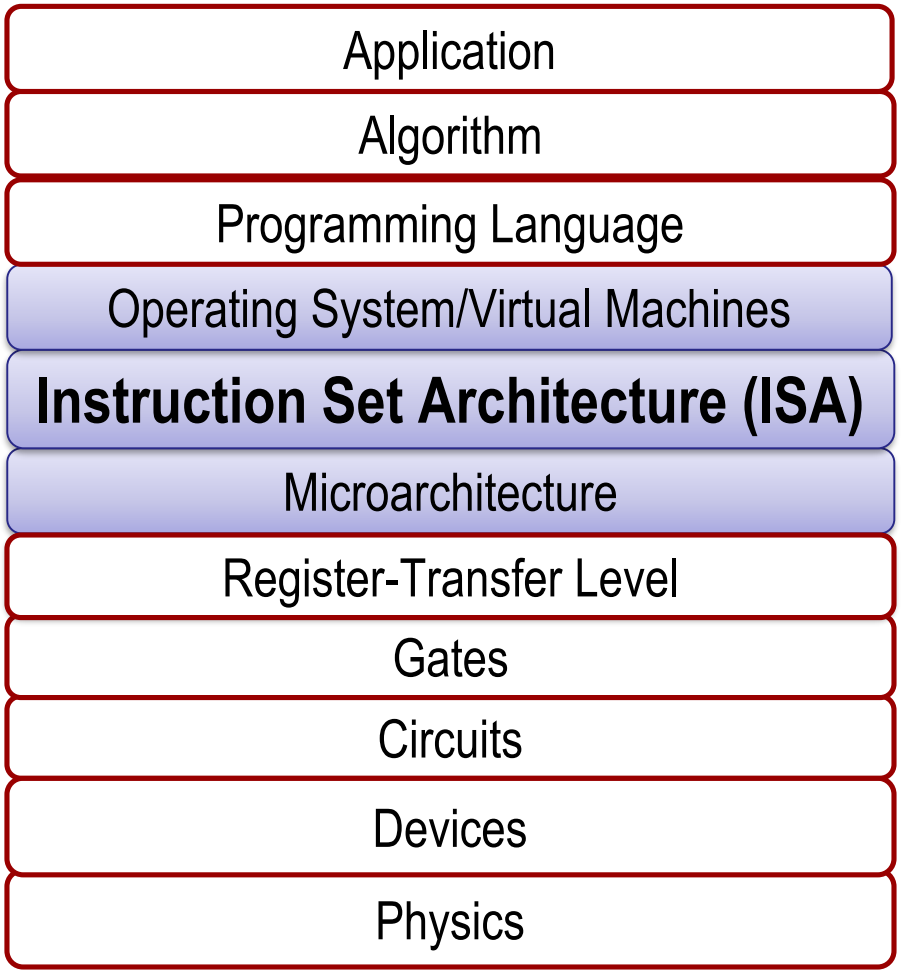
THE UNIVERSITY OF CHICAGO

Machine-Level Programming II: Basics of Instruction Set Architecture (2)

CS144 Fall 2024
Section 3.1-3.5

Yanjing Li

Big Picture



Lecture Goals

- Movq in x86-64
- Memory addressing
- Other basic instructions

(Review) Instruction Set Architecture (ISA)

Programmer

```
void swap (long *xp,
long *yp)
{
  long t0 = *xp;
  long t1 = *yp;
  *xp = t1;
  *yp = t0;
}
```

Compilation (gcc, clang)

```
swap:
    movq (%rdi), %rax
    movq (%rsi), %rdx
    movq %rdx, (%rdi)
    movq %rax, (%rsi)
    ret
```

connect the two

ISA – *interface* between software and hardware

This course uses x86-64 ISA

mulq
xorq
movq
leaq

Hardware (Processor + Memory)

(Review) x86-64 Integer Registers

%rax	%eax	%r8	%r8d
%rbx	%ebx	%r9	%r9d
%rcx	%ecx	%r10	%r10d
%rdx	%edx	%r11	%r11d
%rsi	%esi	%r12	%r12d
%rdi	%edi	%r13	%r13d
%rsp	%esp	%r14	%r14d
%rbp	%ebp	%r15	%r15d

- 64-bit long (word size)
  - Can reference low-order 4 bytes (also low-order 1 & 2 bytes)
- General-purpose registers (except %rsp)

integer general purpose registers - there are only 16!

grey colums are the lower bits

confused about this page

ISA Intro – Moving Data

movq ***Source, Dest***

↳ copy a piece of data

q means 8 bytes, 64 b

- **Operand Types**
  - **Immediate**: Constant integer data
    - Ex., movq $0x100, %rax (good to initialize something with a constant value.)
  - **Register**: One of 16 integer registers
    - Ex. movq %rax, %r13 (move whatever → to here)
    - Copy the value in %rax to %r13
  - **Memory** (the parentheses mean that it refers to our memory location (memory to register transfer))
    - Ex. movq (%rax), %r13
    - Use the value in %rax as a memory address and copy the value from memory to %r13.
    - *More memory address modes later today*

%rax	%r8
%rcx	%r9
%rdx	%r10
%rbx	%r11
%rsi	%r12
%rdi	%r13
%rsp	%r14
%rbp	%r15

study this page

`movq` Operand Combinations

	Source	Dest	Src,Dest	C Analog
`movq`	*Imm*	*Reg*	`movq $0x4,%rax`	`temp = 0x4;`
		Mem	`movq $-147,(%rax)`	`*p = -147;`
	Reg	*Reg*	`movq %rax,%rdx`	`temp2 = temp1;`
		Mem	`movq %rax,(%rdx)`	`*p = temp;`
	Mem	*Reg*	`movq (%rax),%rdx`	`temp = *p;`

`movq` Operand Combinations

	Source	Dest	Src,Dest	C Analog
`movq`	*Imm*	*Reg*	`movq $0x4,%rax`	`temp = 0x4;`
		Mem	`movq $-147,(%rax)`	`*p = -147;`
	Reg	*Reg*	`movq %rax,%rdx`	`temp2 = temp1;`
		Mem	`movq %rax,(%rdx)`	`*p = temp;`
	Mem	*Reg*	`movq (%rax),%rdx`	`temp = *p;`

Cannot do memory-memory transfer with a single instruction

`ex. *p = *q`

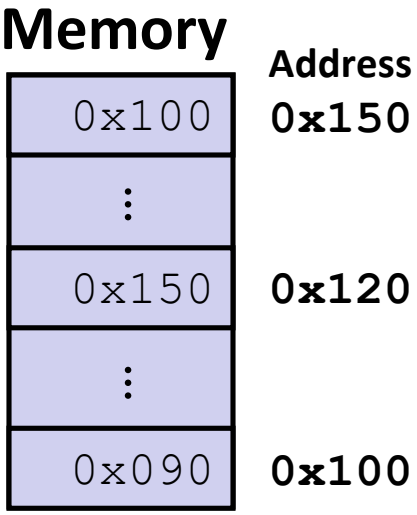
mem-to-mem transfer is two instructions:

example:

```
movq (%rax) %r13
movq %r13 (%rdx)
```

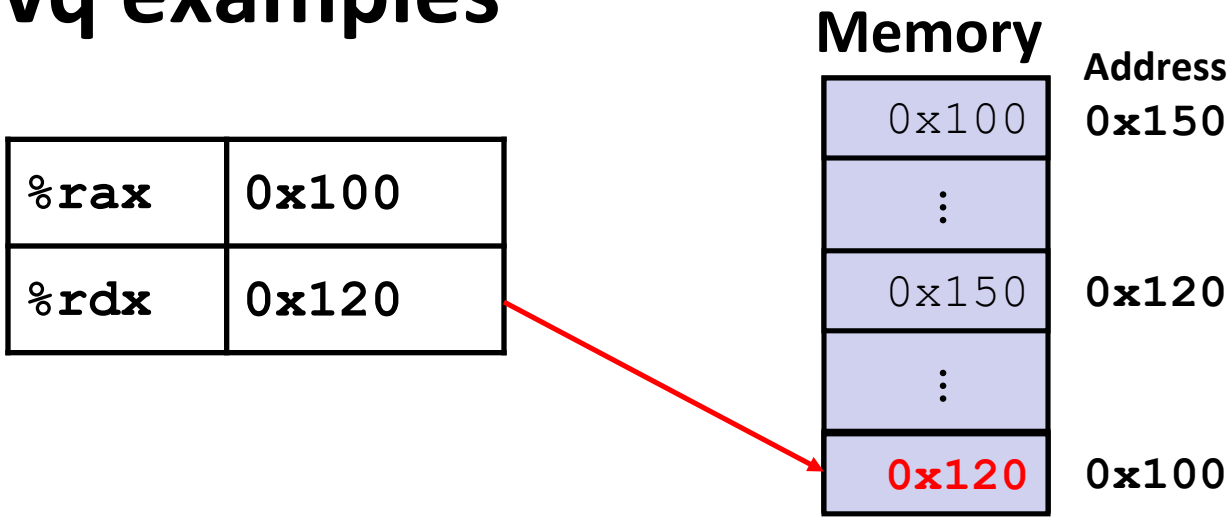
movq examples

%rax	0x100
%rdx	0x120




```
movq $0x120 %rdx
```

movq examples



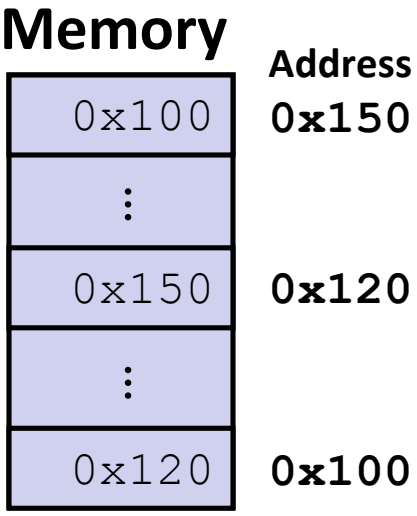

```
movq $0x120 %rdx
```

```
movq %rdx (%rax)
```

we go to the memory and move what was in rdx there

movq examples

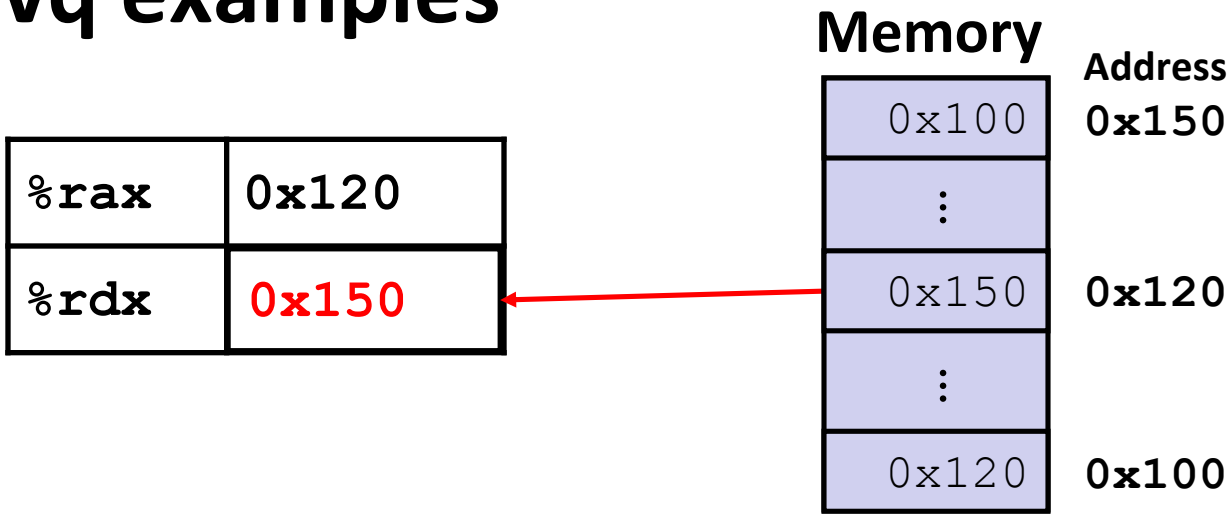
%rax	0x120
%rdx	0x120




```
movq $0x120 %rdx

movq %rdx (%rax)

movq %rdx %rax
```

remember that this is a copy

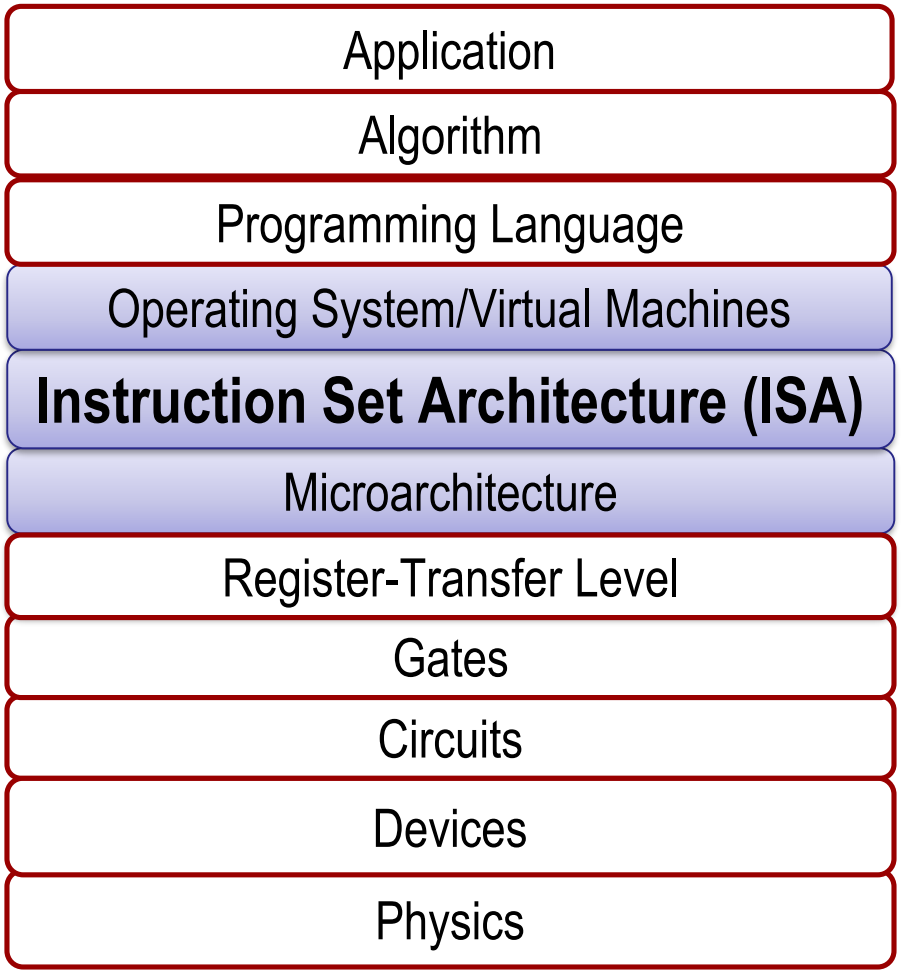
movq examples




```
movq $0x120 %rdx

movq %rdx (%rax)

movq %rdx %rax

movq (%rax) %rdx
```

move the value at 0x120 to rdx

Big Picture



Lecture Goals

- ~~Movq in x86-64~~
- Memory addressing
- Other basic instructions

Simple Memory Addressing Modes

different ways to calculate memory addresses

- **Normal (R) Mem[Reg[R]]**
  - Register R specifies memory address
  - Aha! Pointer dereferencing in C

```
movq (%rcx),%rax     # copy Mem[Reg[rcx]] to rax
```

- **Displacement D(R) Mem[Reg[R]+D]**
  - Register R specifies start of memory region
  - Constant displacement D specifies **offset**

```
movq 8(%rbp),%rdx   # copy Mem[Reg[rbp]+8] to rdx
```

the pointer value in rbp +8

Complete Memory Addressing Modes

- **D(Rb) is a special case of the General Form**

  D(Rb, Ri, S) Mem[Reg[Rb]+S*Reg[Ri]+D]

  - **D**: Constant "displacement", up to 8 bytes in x86-64 (default 0, if omitted)
  - **Rb**: Base register: Any of 16 integer registers (default 0, if omitted)
  - **Ri**: Index register: Any, except for **`%rsp`** (default 0, if omitted)
  - **S**: Scale: 1, 2, 4, or 8 (*to align with typical element size in an array*) (default 1, if omitted)

  Ex. `8(%rax, %rdi, 4)` => Mem addr: `rax + 4*rdi + 8`

- **Special Cases**

  D(Rb) Mem[Reg[Rb]+D] (Displacement)

  (Rb,Ri) Mem[Reg[Rb]+Reg[Ri]]

  D(Rb,Ri) Mem[Reg[Rb]+Reg[Ri]+D]

  (Rb,Ri,S) Mem[Reg[Rb]+S*Reg[Ri]]

  D(,Ri,S) Mem[S*Reg[Ri]+D]

Address Computation Examples

%rdx	0xf000
%rcx	0x0100

D(Rb, Ri, S) Reg[Rb]+S*Reg[Ri]+D

Expression	Address Computation	Address
0x8(%rdx)	0xf000 + 0x8	0xf008
(%rdx,%rcx)	0xf000 + 0x100	0xf100
(%rdx,%rcx,4)	0x f000 + 4·0x100	0xf400
0x80(,%rcx,2)	2· 0x0100 + 0x80	0x0280
0x80(%rcx,%rdx,2)	0x0100 + 2·0xf000 + 0x80	0x1e180

what happens if you go over?

Address Computation Instruction

- **`leaq` *Src*, *Dst* (load effective address)**
  - *Src* is address mode expression
  - Set *Dst* to address denoted by expression

how effective edges

calculate the thing and put it in register

```
leaq 4(%rdi,%rax,2), %rax
```

What would be in `rax`?

Registers

`%rdi`	`0x30`
`%rax`	`0x54`

C analogy:

```
long y = x+y*2+4; // rdi:x, rax:y
```

Difference between `leaq` and `movq`

Unlike `mov`, which copies data at the address src to the destination, `lea` copies the value of src itself to the destination.

```
leaq 4(%rdi,%rax,2), %rax
```

Registers

%rdi	0x30
%rax	0x54

C analogy:

```
long y = x+y*2+4; // rdi:x, rax:y
```

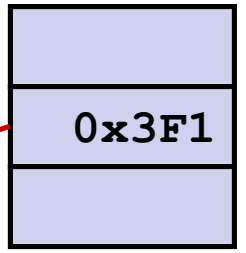
```
movq 4(%rdi,%rax,2), %rax
```

What would be in rax?

Registers

%rdi	0x30
%rax	0x3F1

Memory **Address**



C analogy:

```
y = x+y*2+4;
long temp = *y;
```

movq would calculate, go to memory get that value || leaq just does it and stores it

What's the uses of memory addressing modes

- **Referencing memory location**
  - Refer to an element in an array or struct
  - Refer to a variable in a stack (in a few lectures)

C code

```
int arr[5]={1,2,3,4,5};
return *p = &arr[i];
```

Converted to ASM by compiler:

```
leaq (%rdi,%rsi,4), %rax
```

- **Arithmetic calculation**

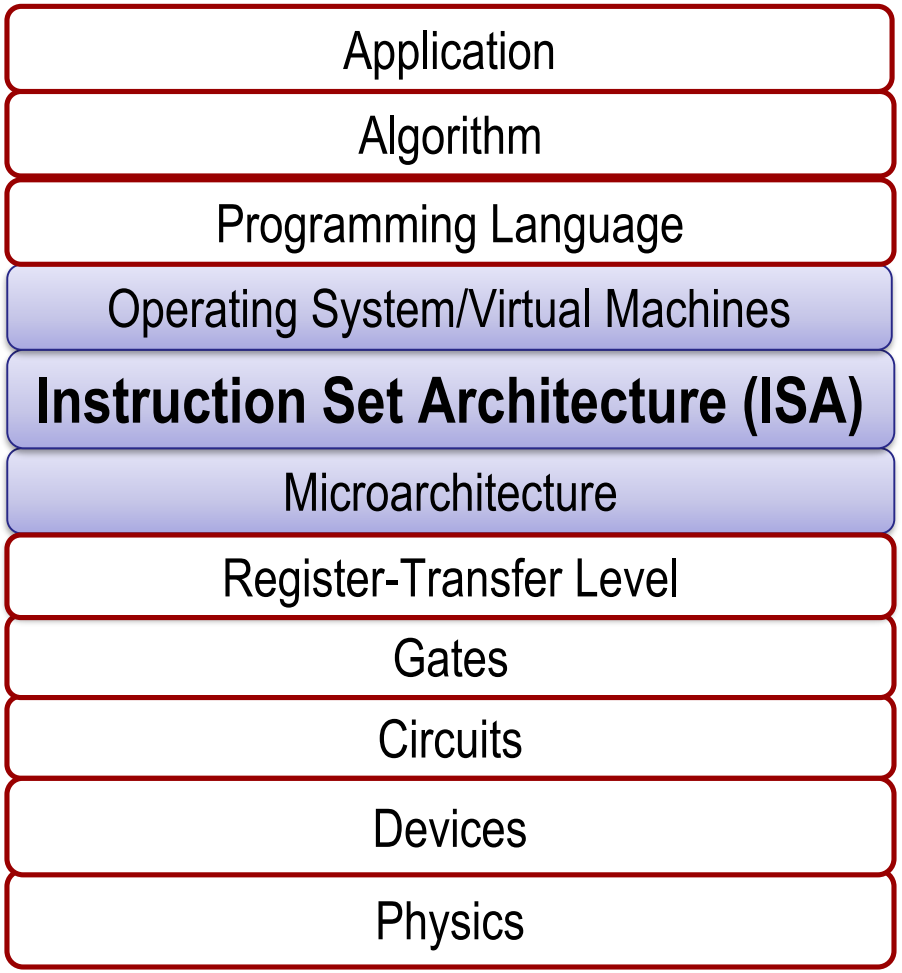
C code

```
long m3(long x)
{
  return x*3;
}
```

Converted to ASM by compiler:

```
leaq (%rdi,%rdi,2), %rax
```

Big Picture



Lecture Goals

- ~~Movq in x86-64~~
- ~~Memory addressing~~
- Other basic instructions

Some Arithmetic Operations

- **Two Operand Instructions:**

Format	*Computation*	
`addq` *Src,Dest*	Dest = Dest + Src	
`subq` *Src,Dest*	Dest = Dest – Src	
`Imulq` *Src,Dest*	Dest = Dest * Src	
`salq` *Src,Dest*	Dest = Dest << Src	***(Also called shlq)***
`sarq` *Src,Dest*	Dest = Dest >> Src	***(Arithmetic)***
`shrq` *Src,Dest*	Dest = Dest >> Src	***(Logical)***
`xorq` *Src,Dest*	Dest = Dest ^ Src	
`andq` *Src,Dest*	Dest = Dest & Src	
`orq` *Src,Dest*	Dest = Dest | Src	

all of these things do something in assembly

bitwise operation

- **Watch out for argument order!**
- **No distinction between signed and unsigned int**

Some Arithmetic Operations

- **One Operand Instructions**

`incq`*Dest*	*Dest = Dest + 1*
`decq`*Dest*	*Dest = Dest – 1*
`negq`*Dest*	*Dest = – Dest*
`notq`*Dest*	*Dest = ~Dest*

Unary operand instructions

- **See book for more instructions**

Computation without a memory reference

C code

```
long m3(long x)
{
  return x*3;
}
```

Converted to ASM by compiler:

```
leaq (%rdi,%rdi,2), %rax # t <- x+x*2
```

```
long m48(long x)
{
  return x*48;
}
```

```
leaq (%rdi,%rdi,2), %rax # t <- x+x*2
salq $4, %rax            # return t<<4
```

e.g.,

- $(1100)_2$ *(2^4)
- $(11000000)_2$

THE UNIVERSITY OF CHICAGO

Extra Materials

How mov works, byte by byte

- **`rax` has `0x0123456789abcdef`, `rdi` has `0x120`, little-endian memory**
  - `movq %rax, (%rdi)`
  - `movq (%rdi), %rdx`
  - `movl (%rdi), %ecx` # **`q`** indicates 8 bytes, and **`l`** indicates 4 bytes
- What if the memory is big endian?
  - `movq %rax, (%rdi)`
  - Will memory layout be different?
  - `movq (%rdi), %rdx`
  - Will **`rdx`** get the same result?
  - `movl (%rdi), %ecx`
  - Will **`rcx`** get the same result? Why?